BCM ASSESSMENT Q2

Assumptions: a WordPress site where one editor cannot upload images but a colleague can.

Likely causes, in the order I would check

1. Role and capability mismatch.
The editor's account may lack the `upload_files` capability. In WordPress this is granted by default to Editors and above, but custom roles or role modifications can remove it.
Check: log in as that user, go to Media, Add New. If the button is missing entirely, it's a capability issue.
Evidence: the admin can upload, this user cannot, and the Media panel looks different.

2. File size or type limit.
The image may exceed the host's `upload_max_filesize` or `post_max_size` PHP setting, or its file type may be blocked. This often looks inconsistent because the colleague is uploading a different image.
Check: try a small JPEG under 2 MB. If it uploads, it's a size or type issue.

3. Disk quota or folder permissions.
If the server's disk is full, or the `wp-content/uploads/` folder lacks write permissions, uploads fail silently or with a generic error.
Check: look at the host's disk usage, and check folder permissions via SFTP or the host's file manager.

Evidence I would look for

The exact error message the editor sees. WordPress errors are often specific and that alone resolves the question.

Once confirmed

- Capability issue: grant `upload_files` via a plugin like User Role Editor, or add the capability programmatically.
- Size or type: raise the PHP limits in `php.ini` or the host's panel.
- Disk or permissions: free disk space, or set directories to 755 and files to 644.

BCM ASSESSMENT Q3

Three improvements I would consider

1. Image optimization and lazy loading.
Large unoptimized images are the most common cause of slow mobile pages.
Measure first: run Lighthouse or PageSpeed Insights and look at Largest Contentful Paint. If images dominate that metric, this is the fix.

2. JavaScript and third-party script audit.
Ads, analytics, and tracking scripts often block rendering on mobile.
Measure: look at the waterfall in Chrome DevTools, or run WebPageTest on a throttled 4G connection to see what runs on page load.

3. Server response time and caching.
If the HTML takes more than a second to arrive, no amount of frontend optimization helps.
Measure: check Time to First Byte in DevTools, and look at the host's cache hit rate.

Chosen improvement

Image optimization and lazy loading.

How I would verify without breaking ads, analytics, or signed-in behaviour

- Set up a staging copy with the changes. Run Lighthouse on staging and production. Compare LCP, TBT, and CLS.
- Load the page with ads blocked, ads enabled, and while signed in. Confirm all three behave identically to production.
- Check the analytics payload in the DevTools Network tab. Confirm the tracking request still fires with the same parameters.
- Check signed-in pages. Confirm the cache doesn't serve a signed-out version to a signed-in user. If the caching layer uses cookies to differentiate, verify the cookie handling is unchanged.
- Roll the change out to 10% of traffic first if possible, and watch Core Web Vitals and error rates before rolling out to all readers.